

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XI
Error Handling



Disusun Oleh:
Intan Rhevita Arselina Suryana (105224009)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. PENDAHULUAN

1.1 Latar Belakang

Dalam dunia rekayasa perangkat lunak, pembuatan program yang berjalan dengan lancar tanpa hambatan adalah impian setiap developer, namun realitanya sebuah kode tidak pernah luput dari kondisi tidak terduga yang dapat menyebabkan kegagalan sistem. Kondisi tidak terduga inilah yang kemudian memicu pentingnya penerapan mekanisme error handling atau penanganan kesalahan, yaitu sebuah proses yang sengaja dirancang untuk mengantisipasi, mendeteksi, dan merespons kegagalan atau kondisi tidak normal saat program sedang dieksekusi. Tanpa adanya sistem penanganan kesalahan yang baik, sebuah aplikasi akan menjadi sangat rapuh karena setiap kali sistem menemui masalah, program akan langsung berhenti bekerja secara paksa tanpa memberikan kejelasan mengenai apa yang sebenarnya terjadi di balik layar.

Secara arsitektural, mekanisme penanganan kesalahan ini biasanya berpusat pada penggunaan blok try dan catch yang bertugas memantau jalannya baris kode secara real-time. Melalui pendekatan ini, sistem diberi kemampuan untuk mencoba menjalankan suatu blok instruksi, dan ketika ada masalah yang muncul di tengah jalan, alur program tidak akan langsung mati melainkan langsung dialihkan ke penurutan instruksi alternatif. Pengalihan ini sangat krusial agar aplikasi tetap bisa berjalan atau minimal memberikan pesan peringatan yang informatif dan mudah dipahami oleh pengguna, bukan sekadar menampilkan baris error sistem yang membingungkan bagi orang awam.

Lebih jauh lagi, pengembangan struktur error handling yang matang juga memungkinkan pengembang untuk membuat exception buatan sendiri atau custom exception demi mengakomodasi kebutuhan logika bisnis yang lebih spesifik. Dengan memanfaatkan custom exception ini, kita bisa mengelompokkan dan membedakan jenis kesalahan secara modular, seperti melakukan validasi format identitas pengguna yang tidak sesuai aturan atau mendeteksi keterbatasan kuota pesanan yang sudah habis. Penerapan strategi penanganan kesalahan yang terstruktur seperti ini pada akhirnya tidak hanya meningkatkan keamanan aplikasi dari sisi pengguna, tetapi juga sangat membantu tim pengembang dalam melakukan proses debugging dan pemeliharaan kode di masa mendatang.

II. VARIABEL

No	Nama Variabel	Tipe Data	Fungsi
1	namaKereta	String	Buat nyimpen nama kereta yang ditampilkan ke pengguna.
2	kodeKereta	String	Buat nyimpen kode unik kereta supaya gampang dicari.
3	rute	String	Buat nyimpen jalur perjalanan kereta.
4	kapasitas	int	Buat nyimpen jumlah kursi yang masih tersedia.
5	daftarKereta	ArrayList<KeretaApi>	Tempat nyimpen semua data kereta yang tersedia.
6	daftar	KeretaApi	Dipakai sementara buat ngambil satu data kereta saat perulangan.
7	kode	String	Buat nampung kode kereta yang diinput penumpang.
8	nik	String	Buat nampung NIK penumpang yang mau pesan tiket.
9	nama	String	Buat nampung nama penumpang.
10	jumlahTiket	int	Buat nampung jumlah tiket yang ingin dipesan.
11	i	int	Counter perulangan saat ngecek setiap karakter pada NIK.
12	a	char	Buat nyimpen satu karakter dari NIK yang sedang dicek.
13	keretaDitemukan	KeretaApi	Dipakai buat nyimpen data kereta yang berhasil ditemukan berdasarkan kode.
14	pesan	String	Parameter yang dipakai buat ngirim pesan error ke Exception.
15	namaKereta	String	Parameter pada TiketHabisException buat nyimpen nama kereta yang kehabisan tiket.
16	sisakursi	int	Parameter pada TiketHabisException buat nyimpen jumlah kursi yang tersisa.
17	input	Scanner	Objek Scanner buat baca semua input dari keyboard.
18	reservasi	Reservasi	Objek utama yang dipakai buat menjalankan fitur pemesanan tiket.
19	jalan	boolean	Penanda apakah menu masih berjalan atau program berhenti.
20	menu	int	Buat nampung pilihan menu yang dipilih pengguna.

21	jumlah	int	Buat nampung jumlah tiket yang diinput pengguna pada menu pemesanan.
22	e	Exception	Variabel penampung objek exception saat terjadi error.

III. DOKUMENTASI DAN PEMBAHASAN CODE

```
public class KeretaApi {
    private String namaKereta;
    private String kodeKereta;
    private String rute;
    private int kapasitas;

    public KeretaApi(String namaKereta, String kodeKereta, String rute, int kapasitas) {
        this.namaKereta = namaKereta;
        this.kodeKereta = kodeKereta;
        this.rute = rute;
        this.kapasitas = kapasitas;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | ✕
    public String getNamaKereta() {
        return namaKereta;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | ✕
    public String getKodeKereta() {
        return kodeKereta;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | ✕
    public String getRute() {
        return rute;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | ✕
    public int getKapasitas() {
        return kapasitas;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | ✕
    public void setKapasitas(int kapasitas) {
        this.kapasitas = kapasitas;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | ✕
    public void tampilkanInfo() {
        System.out.println("Nama Kereta : " + namaKereta);
        System.out.println("Kode      : " + kodeKereta);
        System.out.println("Rute      : " + rute);
        System.out.println("Sisa Kursi : " + kapasitas);
    }
}
```

Pada class KeretaApi ini pertama buat dulu atribut naamKereta (String), kodeKereta (String), rute (String), dan kapasitas (int) dengan mengimplementasikan OOP dan menerapkan encapsulation private untuk melindungi objek agar ga bisa di ubah sembarangan dari luar karena data menyimpan info penting. Makanya semua atributnya di buat menjadi private. Lalu buat constructor barang dengan method public dan lakukan assignment nilai dari parameter input ke atribut class. Kata kunci “this” digunakan untuk merujuk langsung ke atribut milik class itu sendiri. Ini wajib dilakukan karena nama parameter input sama persis dengan nama atribut class. Karena semua atribut

diatur sebagai private, diperlukan metode khusus ber-modifier public untuk mengambil (*read*) nilainya dari luar class. Deklarasi metode Setter dengan nama `setKapasitas(int kapasitas)` untuk mengubah (*write/update*) nilai atribut. Karna cuman ngelakuin pembaruan data dan ga ngembaliin nilai jadi metode ini pake void. Lalu ganti nilai kapasitas lama di dalam objek (`this.kapasitas`) dengan nilai kapasitas baru yang dikirim melalui parameter. Terus `@Override` itu penanda aja buat ngasih tau kompiler Java bahwa metode di bawahnya sedang menggantikan (*override*) metode bawaan yang aslinya milik parent class. Terus bawahnya public void untuk menampilkan info mengenai kapasitas kursi kereta api. Cetak setiap teks beserta nilai dari atributnya masing-masing.

```

import java.util.ArrayList;

Windsurf: Refactor | Explain
public class Reservasi {
    private ArrayList<KeretaApi> daftarKereta;
    public Reservasi() {
        daftarKereta = new ArrayList<>();
        daftarKereta.add(new KeretaApi(namaKereta: "Argo Bromo", kodeKereta: "K01", rute: "JKT - SBY", kapasitas: 50));
        daftarKereta.add(new KeretaApi(namaKereta: "Parahyangan", kodeKereta: "K02", rute: "JKT - BDG", kapasitas: 15));
    }

    Windsurf: Refactor | Explain | Generate Javadoc
    public void tampilkanJadwal() {
        for (KeretaApi daftar : daftarKereta) {
            daftar.tampilkanInfo();
        }
    }

    Windsurf: Refactor | Explain | Generate Javadoc
    public void pesanTiket(String kode, String nik, String nama, int jumlahTiket)
    throws RuteTidakDitemukanException, TiketHabisException {
        if (nik.length() != 16) {
            throw new DataPenumpangTidakValidException(pesan: "NIK harus 16 digit.");
        }

        for (int i = 0; i < nik.length(); i++) {
            char a = nik.charAt(i);
            if (a < '0' || a > '9') {
                throw new DataPenumpangTidakValidException(pesan: "NIK hanya boleh berisi angka.");
            }
        }

        KeretaApi keretaDitemukan = null;
        for (KeretaApi daftar : daftarKereta) {
            if (daftar.getKodeKereta().equalsIgnoreCase(kode)) {
                keretaDitemukan = daftar;
                break;
            }
        }

        if (keretaDitemukan == null) {
            throw new RuteTidakDitemukanException(pesan: "Kode kereta tidak ditemukan.");
        }

        if (jumlahTiket > keretaDitemukan.getKapasitas()) {
            throw new TiketHabisException(keretaDitemukan.getNamaKereta(), keretaDitemukan.getKapasitas());
        }

        keretaDitemukan.setKapasitas(keretaDitemukan.getKapasitas() - jumlahTiket);

        System.out.println(x: "\nPEMESANAN TIKET BERHASIL");
        System.out.println("Nama : " + nama);
        System.out.println("NIK : " + nik);
        System.out.println("Jumlah Tiket : " + jumlahTiket);
    }
}

```

Pada class ini kita harus impor *library* ArrayList dari Java terlebih dahulu agar kita bisa menggunakan struktur data list/daftar yang ukurannya bisa bertambah secara dinamis. Setelah itu buat class *Reservasi* dan buat atribut privat bernama *daftarKereta* yang merupakan sebuah ArrayList. List ini khusus digunakan untuk menampung objek-objek dari class *KeretaApi*. Lalu buat *Constructor* untuk class *Reservasi* dan inisialisasi objek ArrayList kosong baru untuk *daftarKereta* dengan cara *daftarKereta = new ArrayList<>()*. Buat objek kereta baru dengan nama, kode, rute dan kapasitas kursi sesuai yang ditentukan soal, lalu memasukkannya ke dalam list *daftarKereta*. Buat method untuk menampilkan semua jadwal kereta yang tersedia. Gunakan *for-each loop* untuk mengiterasi (mengabsen) setiap objek *KeretaApi* yang ada di dalam *daftarKereta*. Setiap objek yang sedang diakses diwakili

oleh variabel bernama daftar. Kemudian panggil fungsi tampilkanInfo() dari masing-masing objek kereta untuk mencetak detailnya ke layar. Lalu buat method untuk melakukan pemesanan tiket dengan parameter: kode kereta, NIK, nama penumpang, dan jumlah tiket. Method ini juga memberi tahu bahwa ia bisa melempar (*throws*) error/exception tertentu jika terjadi kesalahan aturan bisnis. Cek panjang string nik dengan if (nik.length() != 16), jika jumlah karakternya tidak sama dengan 16, program akan langsung berhenti dan melempar error bahwa NIK tidak valid. Lakukan perulangan untuk memeriksa karakter NIK satu per satu dari indeks pertama sampai terakhir dan ambil karakter pada posisi indeks ke-*i* menggunakan char a = nik.charAt(i); (pakai charAt karena kita mau ngambil 1 karakter dari sebuah String berdasarkan posisi (index)). Cek juga apakah karakter tersebut *bukan* angka. Jika ada satu saja karakter yang bukan angka (0-9), program melempar error "NIK hanya boleh berisi angka". Siapkan variabel keretaDitemukan dengan nilai awal kosong (null) untuk menyimpan kereta yang dicari dan lakukan perulangan untuk mencari kereta di dalam list daftarKereta. Cek apakah kode kereta yang sedang diperiksa sama dengan kode yang diinput oleh pengguna (fungsi equalsIgnoreCase membuat pencarian tidak sensitif terhadap huruf kapital/kecil). Jika kodenya cocok, objek kereta akan disimpan ke variabel keretaDitemukan, lalu perulangan di berhentikan (break). Jika setelah dicari variabel keretaDitemukan tetap null, artinya kode kereta yang dimasukkan salah. Program dihentikan dan melempar error "Kode kereta tidak ditemukan." dan jika jumlah tiket yang diminta lebih besar dari sisa kapasitas/kursi kereta yang ditemukan, program dihentikan dan melempar error bahwa tiket habis atau tidak mencukupi. Nah kalau semua validasi lolos, baru deh jumlah kursi kereta bisa dikurangi. Caranya dengan mengambil kapasitas saat ini (getKapasitas()), dikurangi jumlahTiket yang dipesan, lalu menyimpannya kembali lewat fungsi setKapasitas(). Terakhir cetak struk bukti pemesanan tiket ke layar, menampilkan teks "PEMESANAN TIKET BERHASIL", diikuti dengan Nama, NIK, dan Jumlah Tiket yang dibeli.

```
public class DataPenumpangTidakValidException extends RuntimeException {  
    public DataPenumpangTidakValidException(String pesan) {  
        super(pesan);  
    }  
}
```

Buat Unchecked Exception dengan nama DataPenumpangTidakValidException lalu class ini mewarisi sifat dari RuntimeException. Jadi compiler tidak akan paksa buat pake blok try-catch saat memanggil method yang melempar error ini. Kata kunci super digunakan untuk mengirim parameter pesan (String) ke constructor milik parent class-nya (RuntimeException). Pesan inilah yang akan muncul di konsol saat program error.


```
public class RuteTidakDitemukanException extends Exception {
    public RuteTidakDitemukanException(String pesan) {
        super(pesan);
    }
}
```

Buat juga Checked Exception dengan nama RuteTidakDitemukanException class ini mewarisi langsung class Exception. Karakteristiknya adalah compiler wajib memeriksa penanganannya. Oleh karena itu, di class Reservasi sebelumnya, method pesanTiket harus ditambahkan tulisan throws RuteTidakDitemukanException dan saat dipanggil nanti wajib dibungkus dalam blok try-catch. Baris super(pesan); mengirim teks detail error ke parent class (Exception) agar bisa ditampilkan saat program crash atau ditangkap oleh catch.

```
public class TiketHabisException extends Exception {
    public TiketHabisException(String namaKereta, int sisaKursi) {
        super("Tiket kereta " + namaKereta + " tidak mencukupi. " + "Sisa kursi: " + sisaKursi);
    }
}
```

Buat juga Checked Exception satu lagi dengan nama TiketHabisException class ini mewarisi langsung class Exception. Exception ini membawa info spesifik soal nama kereta dan sisa kursi yang tersedia. Pada bagian terakhir, program akan otomatis nyusun pesan kalau tiket kereta yang di beli tidak cukup dan akan menampilkan jumlah sisa kursi yang tersedia.

```

import java.util.*;

Windsurf: Refactor | Explain
public class Main {
    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Reservasi reservasi = new Reservasi();
        boolean jalan = true;

        while (jalan) {
            try {
                System.out.println(x: "\nMENU JAVA EXPRESS");
                System.out.println(x: "1. Lihat Jadwal");
                System.out.println(x: "2. Pesan Tiket");
                System.out.println(x: "3. Keluar");
                System.out.print(s: "Pilih : ");

                int menu = input.nextInt();
                input.nextLine();

                switch (menu) {
                    case 1:
                        reservasi.tampilkanJadwal();
                        break;

                    case 2:
                        System.out.print(s: "Kode Kereta : ");
                        String kode = input.nextLine();

                        System.out.print(s: "NIK : ");
                        String nik = input.nextLine();

                        System.out.print(s: "Nama : ");
                        String nama = input.nextLine();

                        System.out.print(s: "Jumlah Tiket : ");
                        int jumlah = input.nextInt();
                        input.nextLine();

                        reservasi.pesanTiket(kode, nik, nama, jumlah);
                        break;

                    case 3:
                        jalan = false;
                        break;

                    default:
                        System.out.println(x: "Menu tidak tersedia.");
                }

            }

            catch (InputMismatchException e) {
                System.out.println(x: "Input harus berupa angka.");
                input.nextLine();
            }

            catch (DataPenumpangTidakValidException e) {
                System.out.println("Terjadi kesalahan pada data penumpang: " + e.getMessage());
            }

            catch (RuteTidakDitemukanException e) {
                System.out.println("Terjadi kesalahan pada rute: " + e.getMessage());
            }

            catch (TiketHabisException e) {
                System.out.println("Terjadi kesalahan pada tiket: " + e.getMessage());
            }

            finally {
                System.out.println(x: "Proses selesai.");
            }
        }

        input.close();
    }
}

```

Pada class Main ini perlu ngimpor seluruh class dan interface yang berada di dalam paket java.util dan biar simple bisa menggunakan java.util.*. Buat objek input untuk membaca inputan dari user dengan Scanner input = new Scanner(System.in);. Buat juga objek reservasi baru. Otomatis memicu pengisian daftar kereta awal (Argo Bromo dan Parahyangan) yang diatur di dalam constructor class Reservasi. Pada bagian boolean jalan = true selama nilainya true, menu aplikasi akan terus ditampilkan kembali. Gunakan perulangan while agar variable jalan melakukan perulangan terus menerus hingga nilai berubah false. Java akan memantau seluruh kode di dalam blok try-catch. Jika terjadi crash atau exception di tengah jalan (misalnya user salah ketik input atau tiket habis), program tidak akan langsung mati, melainkan langsung lompat ke blok catch. Lalu cetak menu pilihan ke layar dan baca angka pilihan menu yang diketik oleh pengguna dan menyimpannya ke variabel menu. Gunakan input.nextLine(); untuk menghilangkan buffer dan buang enter agar input berikutnya ga di lewati secara otomatis. Gunakan fungsi switch-case untuk mengecek nilai variable menu. Jika user mengetik angka 1 maka akna dialihkan ke case 1, program memanggil reservasi.tampilkanJadwal(); untuk memunculkan daftar kereta dan sisa kuota kursinya. Setelah itu, fungsi dihentikan dengan perintah break;. Jika user mengetik angka 2 maka akna dialihkan ke case 2, program akan meminta input dan setelah seluruh data terkumpul, program mengeksekusi proses utama dengan memanggil fungsi reservasi.pesanTiket(kode, nik, nama, jumlah);. Jika user mengetik angka 3 maka akna dialihkan ke case 3, status variabel jalan diubah menjadi false. Efeknya, perulangan while akan berhenti pada putaran berikutnya, dan aplikasi selesai. Sedangkan default digunakan saat user mengetik angka lain selain 1,2,3 dan akan mencetak teks "Menu tidak tersedia.". catch (InputMismatchException e) digunakan untuk menangkap error jika pengguna diminta memasukkan angka (pada menu atau jumlah tiket), tetapi malah mengetik huruf. Program akan menampilkan "Input harus berupa angka." dan membersihkan sisa inputan bermasalah tersebut via input.nextLine();. Lalu catch (DataPenumpangTidakValidException e) digunakan jika format NIK penumpang salah (kurang/lebih dari 16 digit atau mengandung huruf). Pesan error spesifiknya diambil menggunakan fungsi e.getMessage(). Lalu catch (RuteTidakDitemukanException e) digunakan jika user salah mengetikkan kode kereta yang tidak ada dalam daftar kereta sistem. Terakhir catch (TiketHabisException e) digunakan jika jumlah tiket kereta yang dipesan melampaui sisa kapasitas kursi yang tersedia. Blok try-catch ini ditutup dengan finally, blok opsional khusus yang pasti akan selalu dijalankan di setiap akhir putaran menu, baik ketika proses pemesanan sukses berjalan dengan lancar maupun saat program baru saja menangkap sebuah error. Blok finally ini akan mencetak pesan "Proses selesai.". input.close terletak di bagian paling terakhir karena berfungsi untuk menutup objek scanner.

IV. KESIMPULAN

Kesimpulan yang saya dapatkan dari praktikum Error Handling ini adalah saya menjadi lebih memahami pentingnya penanganan kesalahan (exception handling) dalam pembuatan aplikasi agar program tetap berjalan dengan stabil meskipun terjadi kesalahan saat proses input maupun eksekusi. Melalui pembuatan sistem pemesanan tiket kereta api "JAVA EXPRESS", saya dapat mempraktikkan penggunaan blok try-catch-finally untuk menangani berbagai kondisi error tanpa membuat program langsung berhenti.

Selain itu, saya juga memahami perbedaan antara Checked Exception dan Unchecked Exception. Checked Exception digunakan untuk kondisi yang wajib ditangani oleh programmer, seperti `RuteTidakDitemukanException` dan `TiketHabisException`, sedangkan Unchecked Exception digunakan untuk kesalahan yang tidak wajib diperiksa oleh compiler, seperti `DataPenumpangTidakValidException`. Praktikum ini juga membantu saya memahami cara membuat custom exception sesuai kebutuhan logika program, melakukan validasi data pengguna, serta menerapkan error handling yang baik agar aplikasi lebih aman, mudah dipelihara, dan memberikan pesan kesalahan yang lebih jelas kepada pengguna.

V. DAFTAR PUSTAKA

[Java Exception Handling - GeeksforGeeks](#)